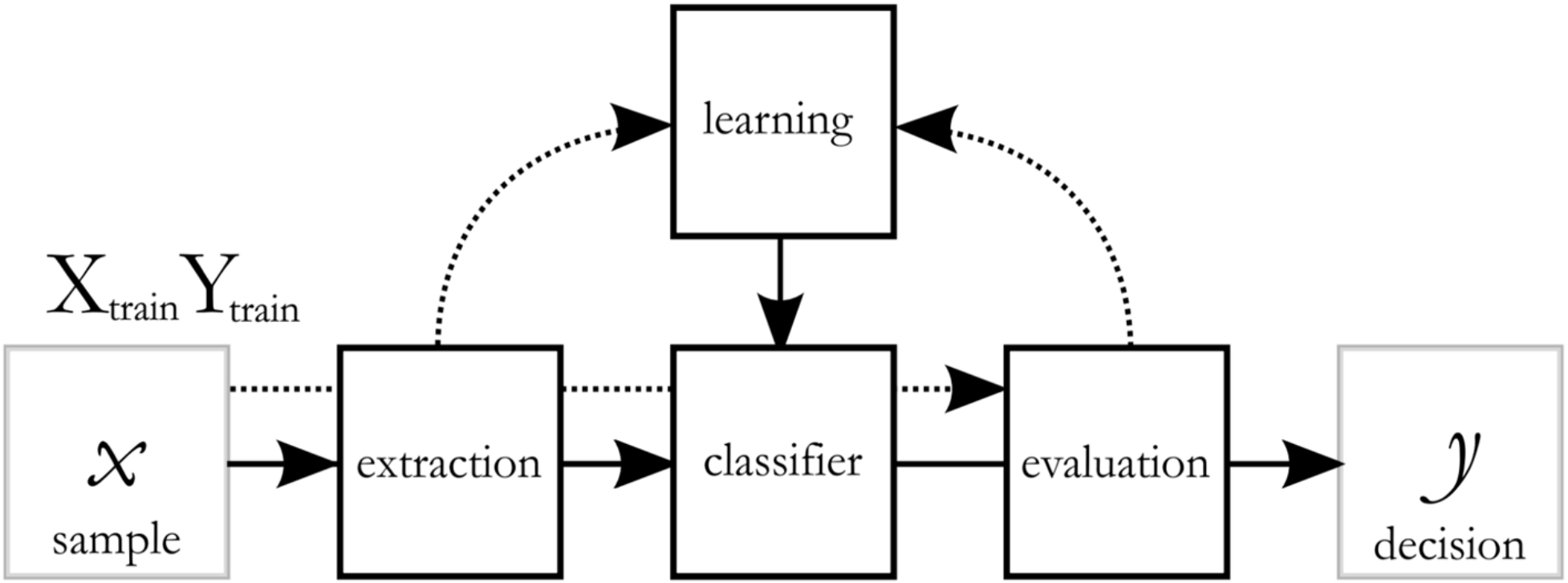
FACULDADE DE CIÊNCIAS E TECNOLOGIA
DEPARTAMENTO DE FÍSICA

Machine Leaning (ML)

LINEAR MODELS – LINEAR, POLYNOMIAL AND LOGISTIC REGRESSION

Pedro Vieira
pmv@fct.unl.pt

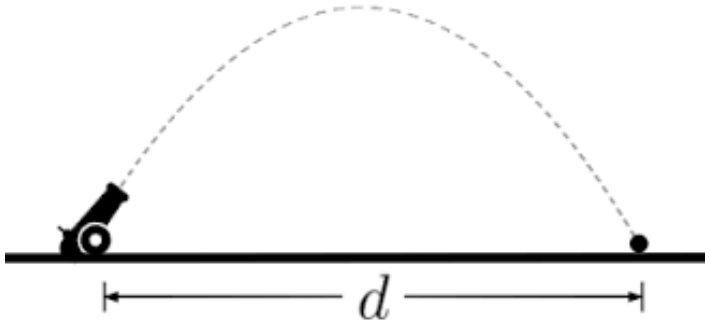
Linear Models



Linear Models
Linear Regression

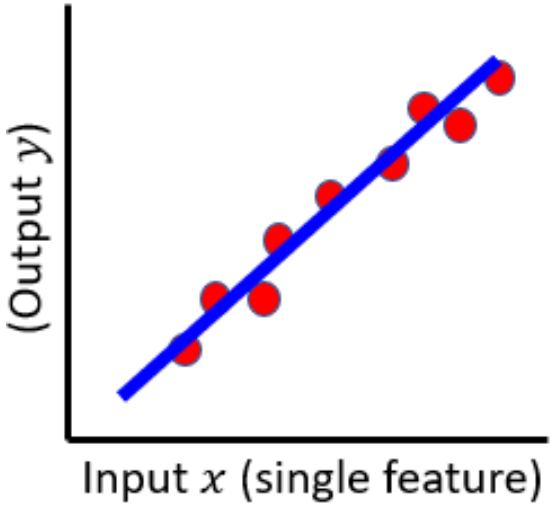
Linear regression is a supervised learning model that tries to predict a continuous value (a number) by fitting the best straight line through your data

Example: In ballistic problem what is the range of the projectile if we know the initial velocity and initial angle constant.



Experimental data only

- X (feature) – inicial velociy
- Y (decision) - range



Linear Models
Linear Regression

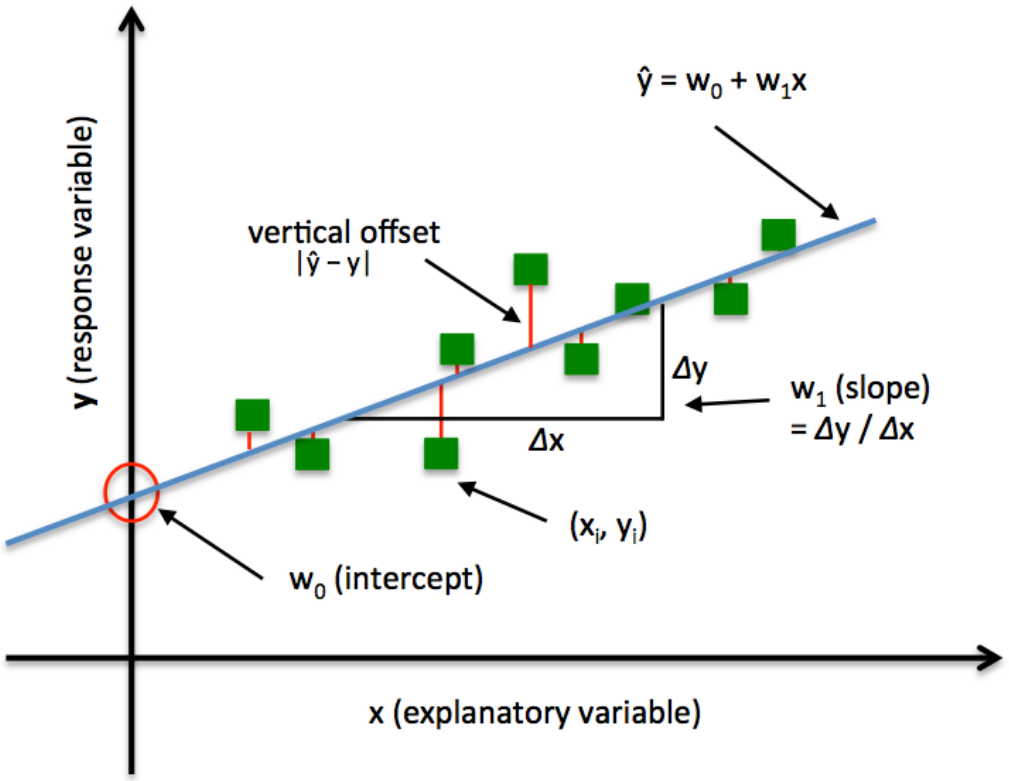
This is the base model for all statistical machine learning

- x is a one feature data variable
- y is the value we are trying to predict

The regression model is

$$y = w_0 + w_1 x + \varepsilon$$

- Two parameters to estimate – the slope of the line w1 and the y-intercept w0
- ε is the unexplained, random, or error component.



Linear Models
Linear Regression

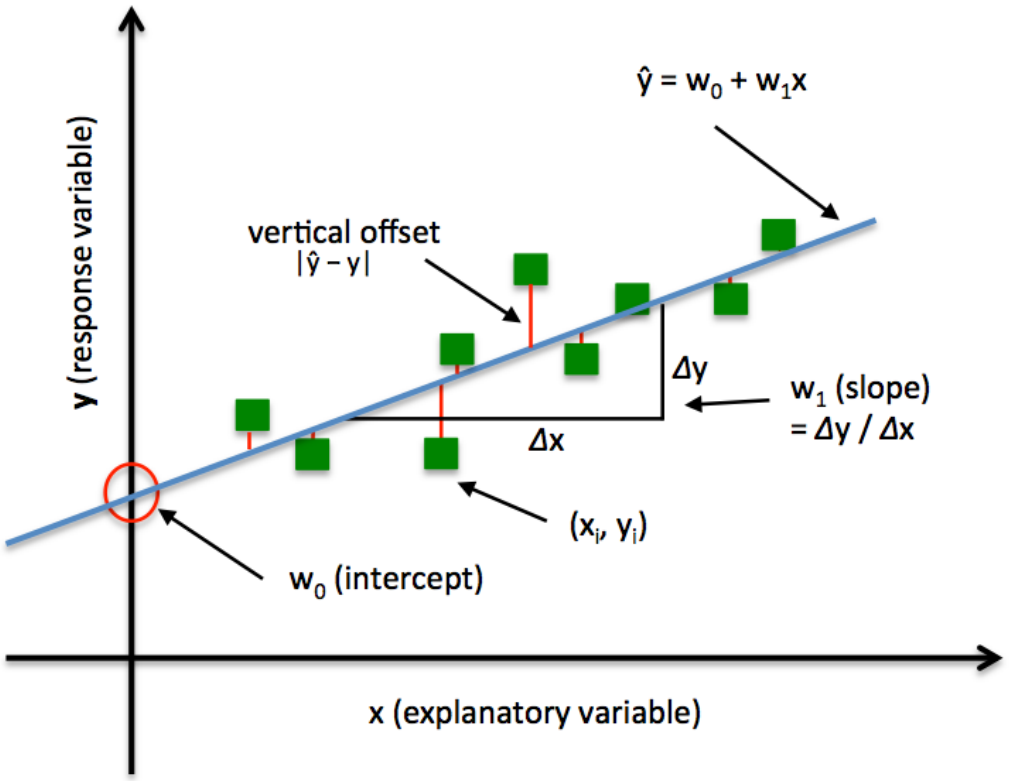
We basically want to find {w0, w1} that minimize deviations from the predictor line

$$\arg\min_{w_0,w_1} \sum_{i}^{n} (y_i - w_0 - w_1 x_i)^2$$

How do we do it?

- Iterate over all possible w values along the two dimensions (bruit force)?
- Find an analytical solution (closed form) with just plain calculus
- But, very few optimization problems in ML have analytical solutions

This problem has an analytical solution, but what happens if we have more features, as velocity, resistance of air, temperature, etc…



Linear Models
Linear Regression

Suppose that you have as a second feature, the initial angle.

- Now you need to fit a plan

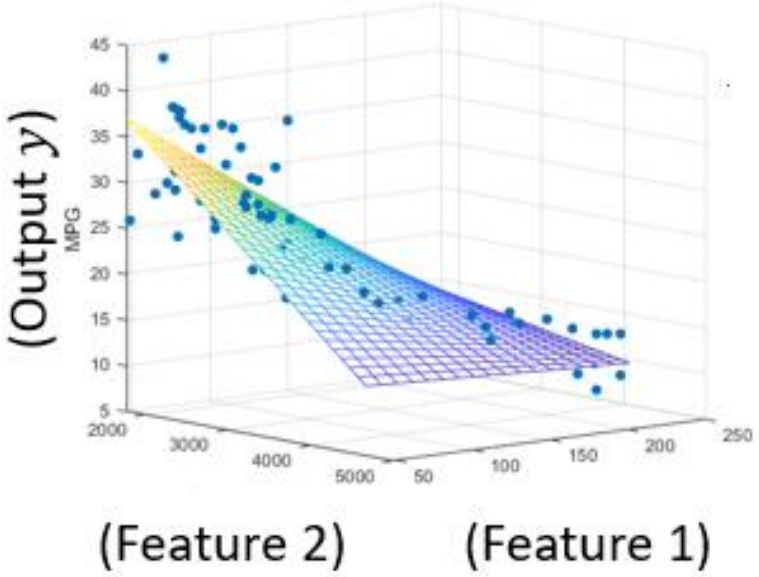
$$y_i = \beta_0 + \beta_1 x_i + \varepsilon_i,\ \text{i=1,2,...,n}$$

And we what to minimise:

$$\sum_{i=1}^{n} \varepsilon_i^2 = \sum_{i=1}^{n} (y_i - \beta_0 - \beta_i x_i)^2$$

We assume that

- The features are independent.
- the errors are normal, mutually independent



Linear Models
Linear Regression

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Where:

- $\hat{y}$ is the predicted value
- n is the number of features
- x_i is the ith feature
- θ_i is the parameter to optimise

The most used measurement of error is the Mean Square Error (MSE)

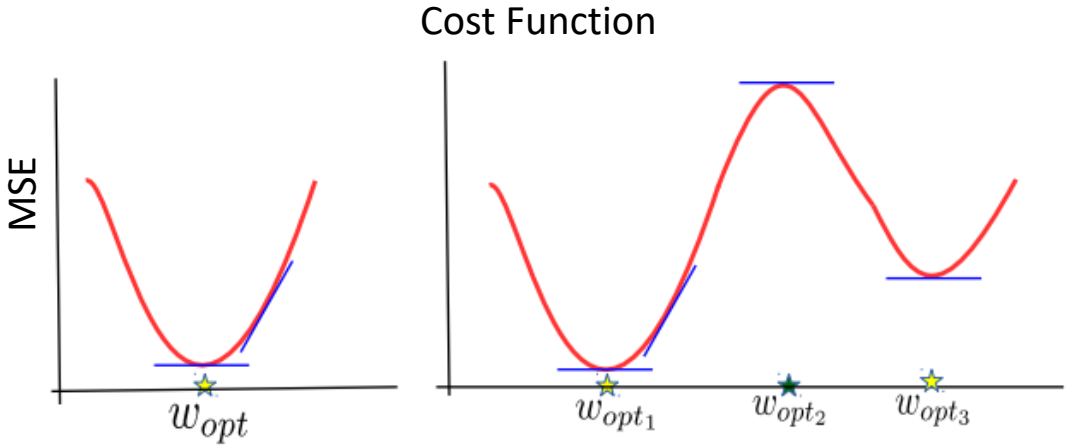
$$\mathrm{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)} \right)^2$$

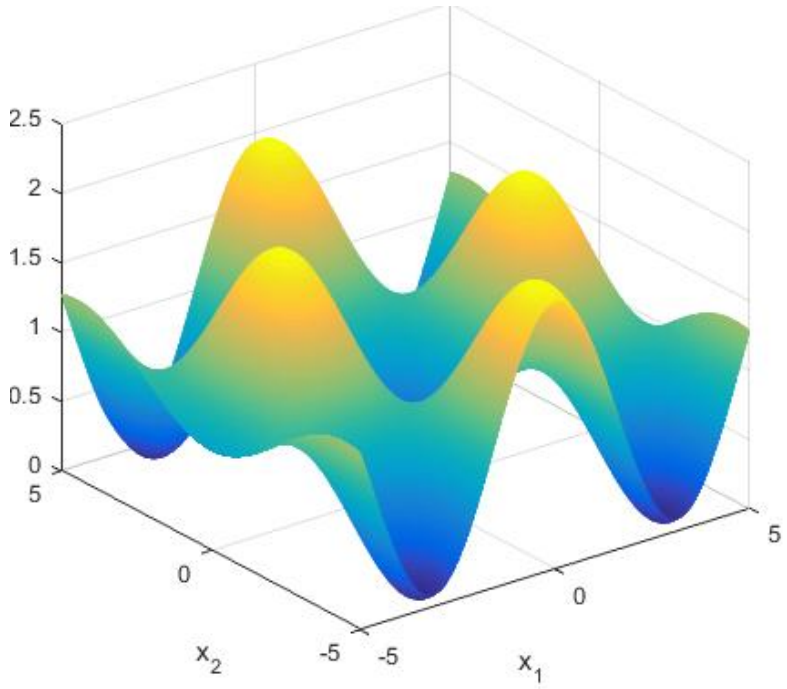
Linear Models

Linear Regression - Gradient Descent Algorithm

The goal is to adjust the θ_i in order to minimise the MSE

But, this could be a complex problem due to the complex form of MSW vs θ_i (w_i)





One way to solve the problem is using the **Gradient Descent Algorithm**

Linear Models
Linear Regression - Gradient Descent Algorithm

The base of the gradient descent algorithm is:
An optimal solution is found if gradient = 0

MSE:

$$\mathrm{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}\right)^2$$

Gradient:

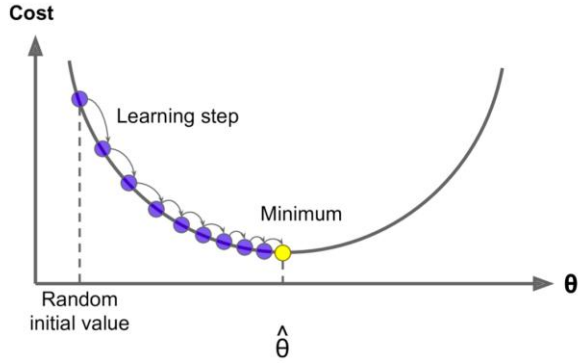
$$\frac{\partial}{\partial \theta_j} \mathrm{MSE}(\boldsymbol{\theta}) = \frac{2}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}\right) x_j^{(i)}$$

The algorithm:

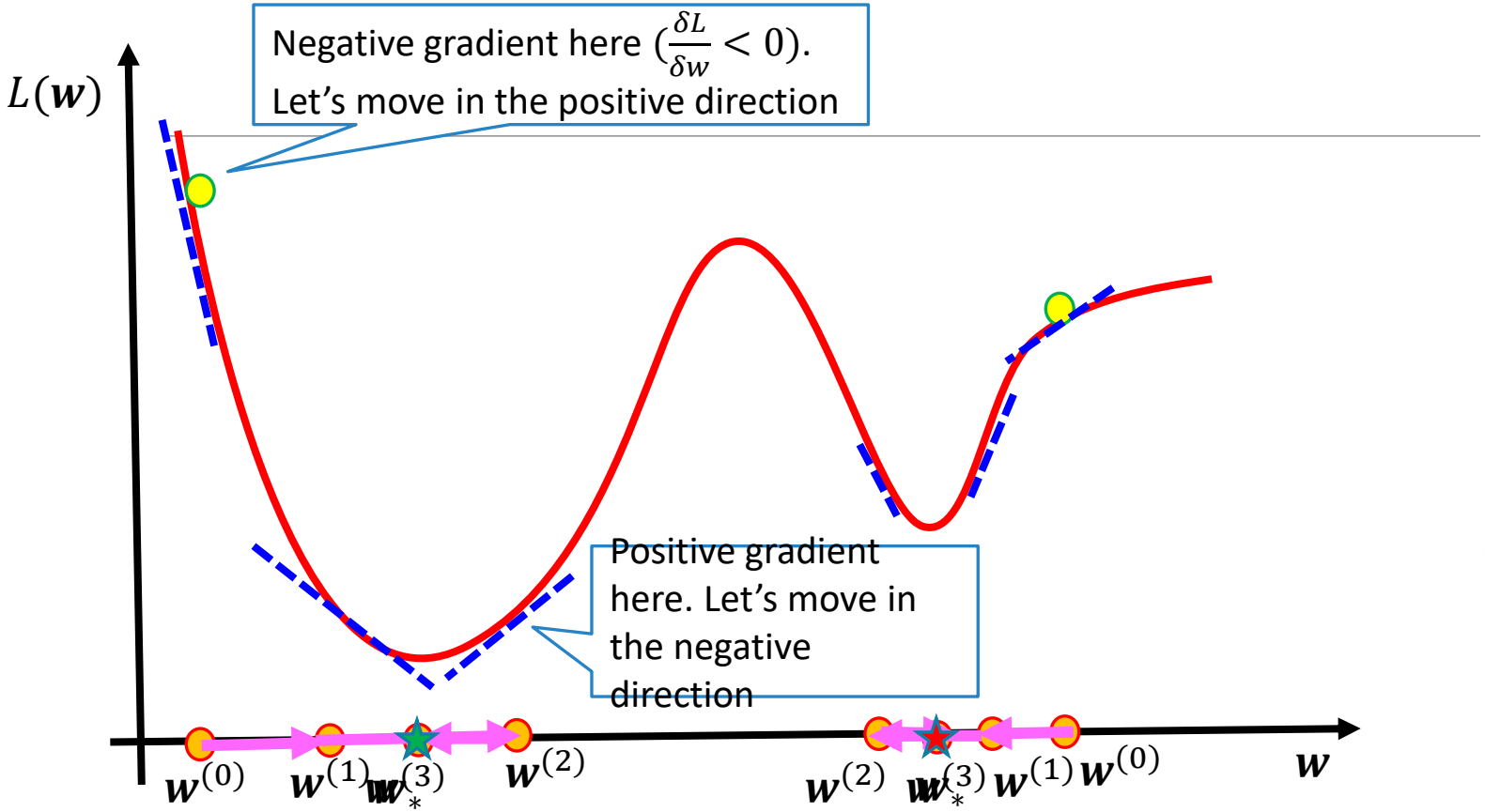
1. Initialize θ_i
2. Set a learning rate η_i
3. Calculate the gradient ∇_θ with current θ_i
4. Move in the opposite direction of the gradient

$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathrm{MSE}(\boldsymbol{\theta})$$

5. Repeat from 3 until solution found



Linear Models - Linear Regression



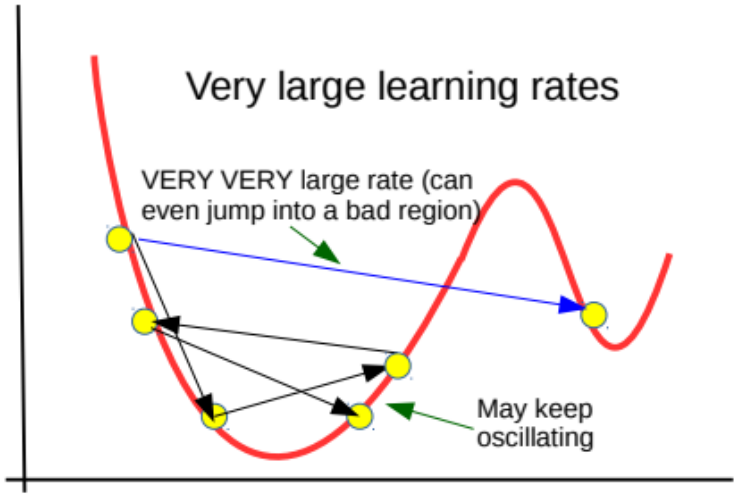
Woohoo! ☺
Global minima found!!!

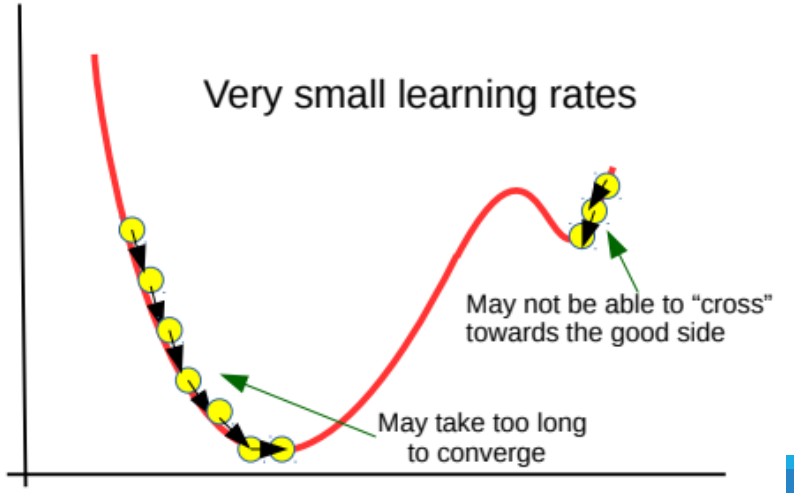
GD thanks you for the good initialization ☺

Stuck at a local minima ☹

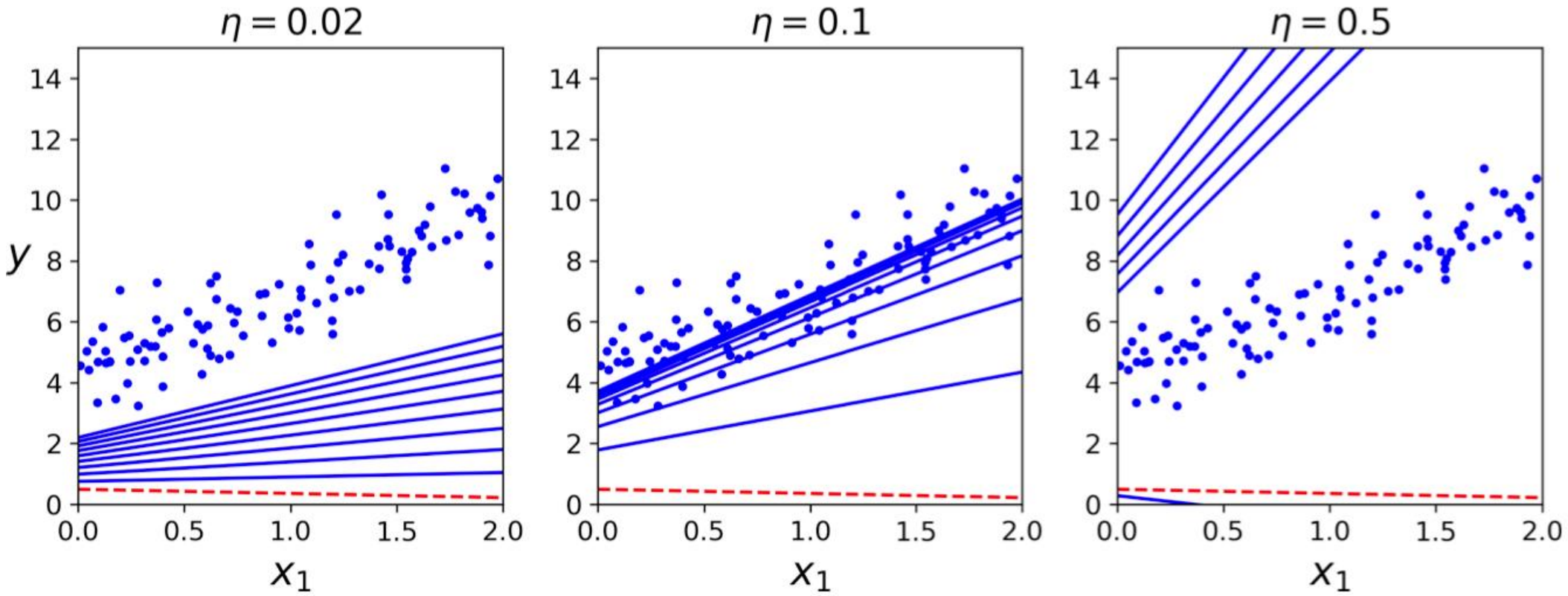
Good initialization is very important

Learning rate is very important





Linear Models
Linear Regression



Influence in the convergence with the learning rate.

Linear Models
Linear Regression

A fixed learning rate can make you fail the solution.

Algorithms like:

- AdaGrad
- RMSProp
- Adam

Adjust the learning rate automatically based on the history of gradients.

Example: in AdaGrad:

η é substituído por: $\eta = \frac{\alpha}{\sqrt{\sum(\nabla)+\varepsilon}}$

Where:

- α – step size
- $\sum(\nabla)$ - sum of the previous gradiants
- ε – stability constant

This result in:

- Parameters that receive large gradients get smaller learning rates.
- Parameters that receive small gradients get larger learning rates.
- The leaning rate become smaller when near minimum

Linear Models
Linear Regression

Do's

- When the relationship is roughly straight-line
- When features are independent
- When the data is not too noisy
- When interpretability matters (you want to understand the model)

Dont's

- Non-linear patterns
- Strong interactions between features
- Outliers
- Very high-dimensional data

Linear Models
Regularized Linear Regression

In the situation of data with too much noise, significate number of outliers or too many features, can result in a bad fitting due to the model to give too much weight to some variables. This result in:

- bad predictions on new data
- unstable coefficients
- sensitivity to small changes

The solution: penalized features the have large weights θ

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

Mean Square Error

$$\mathrm{MSE}(\mathbf{X}, h_{\boldsymbol{\theta}}) = \frac{1}{m} \sum_{i=1}^{m} \left(\boldsymbol{\theta}^T \mathbf{x}^{(i)} - y^{(i)}\right)^2$$

Ridge Regression:

$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + \alpha \frac{1}{2} \Sigma_{i=1}^{n} \theta_i^2$$

Lasso Regression:

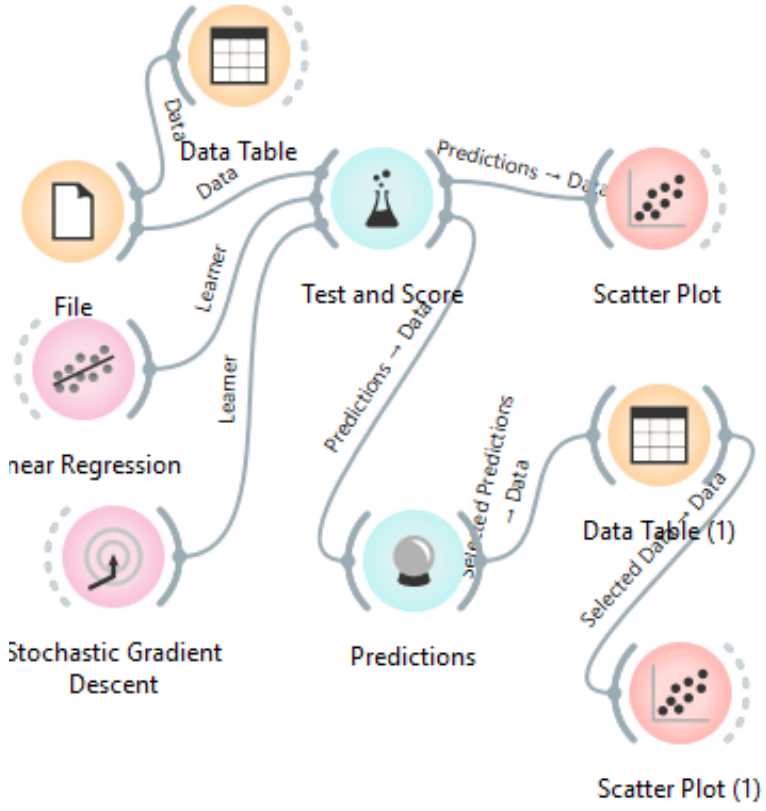
$$J(\boldsymbol{\theta}) = \mathrm{MSE}(\boldsymbol{\theta}) + \alpha \Sigma_{i=1}^{n} \left|\theta_i\right|$$

Use *J* in gradient descent instead of *MSE*

$$\boldsymbol{\theta}^{(\text{next step})} = \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} \mathrm{MSE}(\boldsymbol{\theta})$$

Linear Models
Linear Regression

Feature	Type	Meaning
CRIM	numeric	Per-capita crime rate by town
ZN	numeric	Proportion of residential land zoned for large lots
INDUS	numeric	Proportion of non-retail business acres
CHAS	categorical (0/1)	Charles River dummy variable (1 = tract bounds river)
NOX	numeric	Nitric oxide concentration (ppm)
RM	numeric	Average number of rooms per dwelling
AGE	numeric	Proportion of owner-occupied units built before 1940
DIS	numeric	Weighted distances to employment centers
RAD	numeric	Index of accessibility to radial highways
TAX	numeric	Property tax rate per $10,000
PTRATIO	numeric	Pupil-teacher ratio by town
B	numeric	$1000(Bk - 0.63)^2$ where Bk is proportion of Black residents
LSTAT	numeric	% of lower-status population
MEDV	numeric (target)	Median value of owner-occupied homes (in $1000s)



Linear Models
Polynomial Regression

Until now we have considered a first order fitting function, but it can be generalised to a polynomial function of any order.

$$Y_i = \beta_0 + \beta_1 X_i + \beta_2 X^2{}_i + \varepsilon_i$$

It is possible that you get a better fit of your data, but at the price of having more variables to optimise.

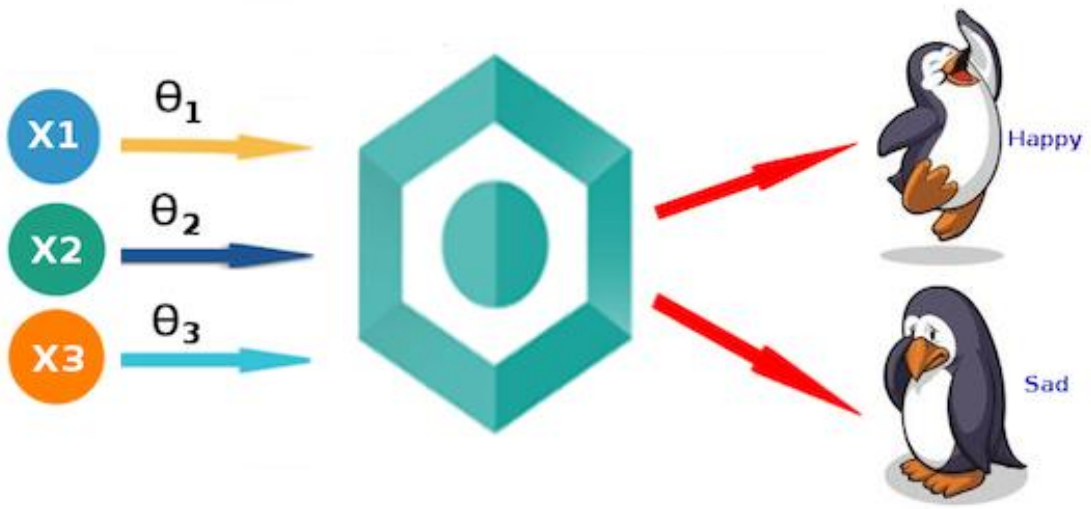
Linear Models
Logistic Regression

Logistic Regression predicts probabilities, not continuous values, as the linear regression

It generates a binary classification, with a probability threshold normally set to 0.5

Example of applications:

- Prediction of phase transitions
- Prediction of the stability of a plasma in tokamaks
- Prediction of climacteric conditions
- Prediction of a tumor is malignant?



Given these inputs, what is the probability that the outcome is 1?

Linear Models
Logistic Regression

In linear regression models, the prediction output could any number.

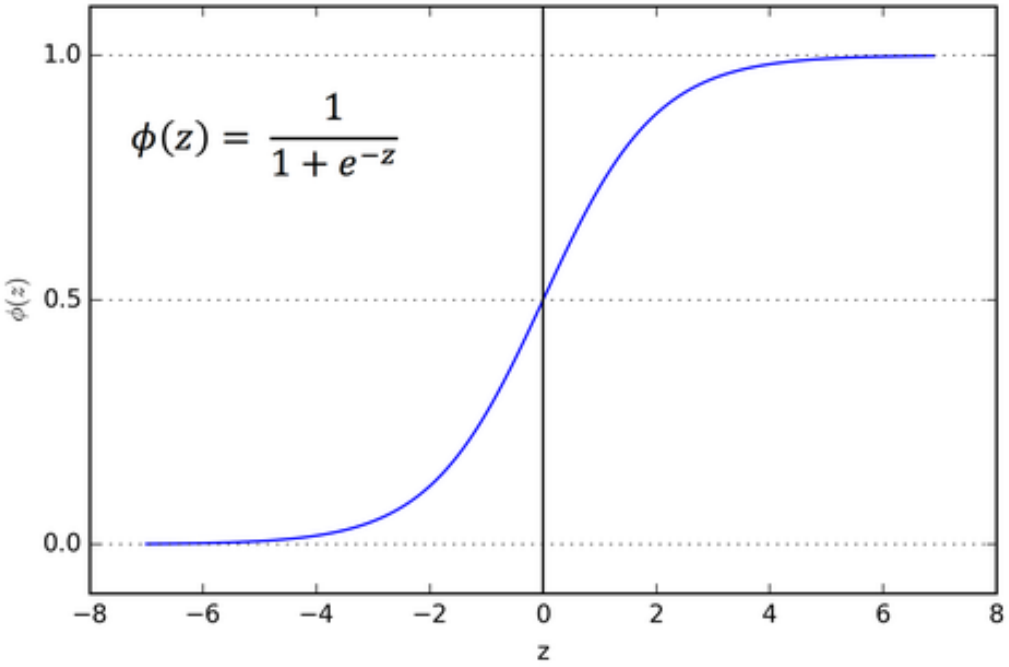
To convert the prediction for a probability between 0 and 1, it is used a **sigmoid function**

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$

- something close to 0 when t is very negative
- something close to 1 when t is very positive
- something around 0.5 when t ≈ 0

It models the soft transition between states, that mimic many process in nature.

Many physics process are described by identical curves as the Fermi-Dirac distribution and Boltzman factor.



Linear Models
Logistic Regression

Linear regression model (continuous output)

$$\hat{y} = \theta_0 + \theta_1 x_1 + \theta_2 x_2 + \cdots + \theta_n x_n$$

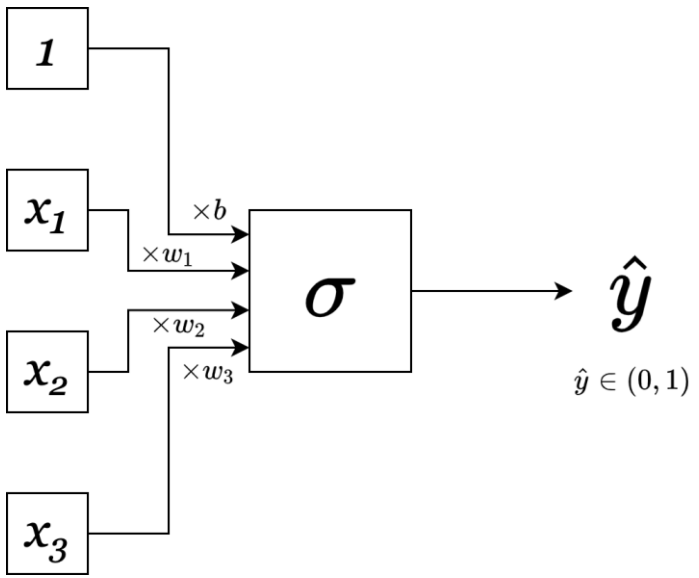
Logistic regression (probabilistic):

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x}) = \sigma\left(\mathbf{x}^T \boldsymbol{\theta}\right)$$

$$\sigma(t) = \frac{1}{1 + \exp(-t)}$$

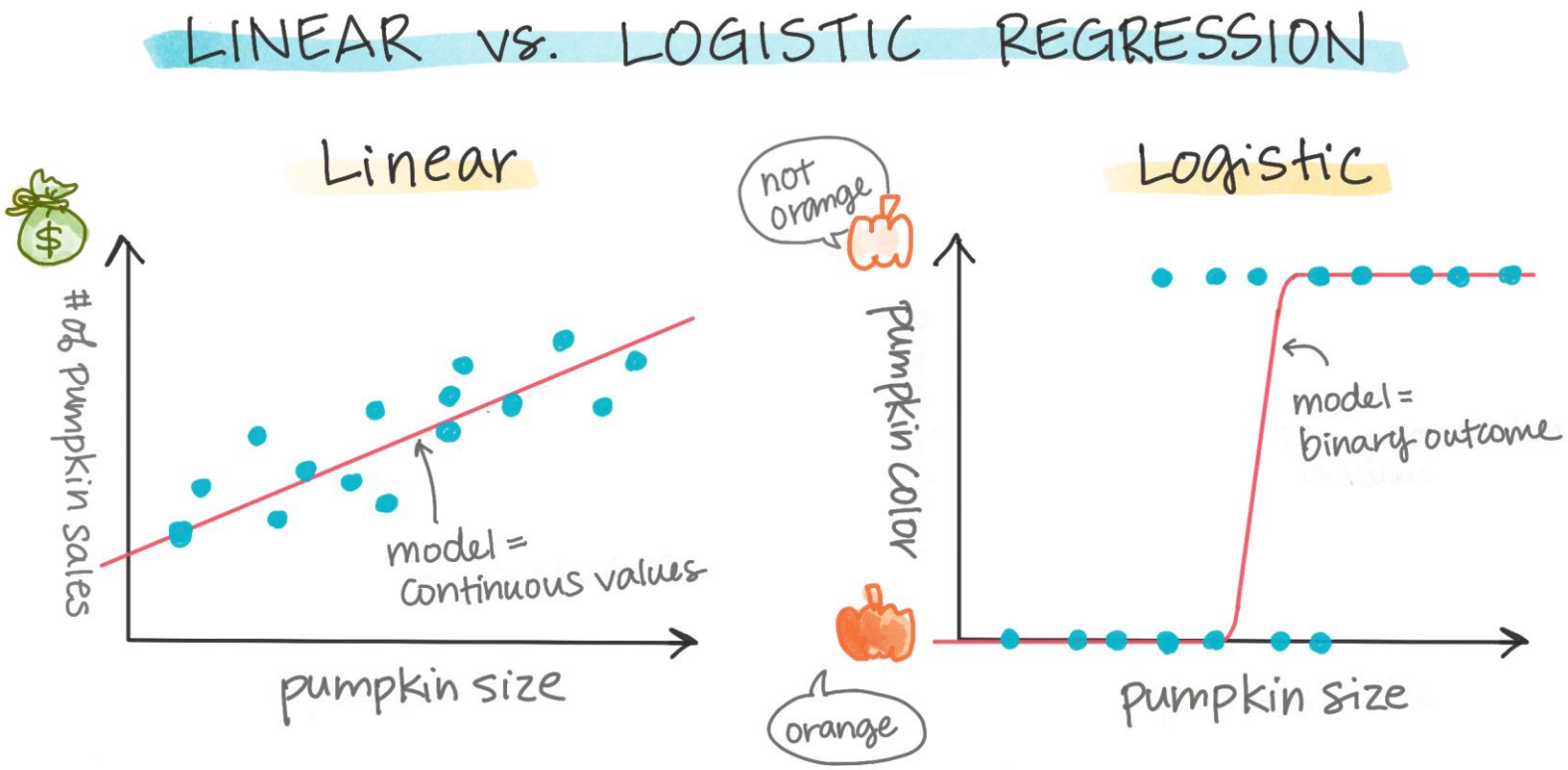
$$\hat{y} = logistic(\hat{b} + \widehat{w_1} \cdot x_1 + \cdots + \widehat{w_n} \cdot x_n)$$

$$= \frac{1}{1 + exp[-(\hat{b} + \widehat{w_1} \cdot x_1 + \cdots + \widehat{w_n} \cdot x_n)]}$$



$\hat{y}$ represents a probability of an outcome with values [0,1]

Linear Models
Logistic Regression



Linear Models
Logistic Regression

Which metrics to use to know how well the model predicted probabilities, match the actual data.

- By using the NLL - negative log-likelihood

We know that the probability of obtain an outcome is the multiplication of all individual probabilities:

$$p(\boldsymbol{y}|\boldsymbol{w},\boldsymbol{X}) = \prod_{n=1}^{N} p(y_n|\boldsymbol{w},\boldsymbol{x}_n)$$

We can convert the multiplication to a sum by doing -Log(p). (-) to make a problem of maximization and not minimization

So:

$$c(\boldsymbol{\theta}) = \begin{cases} -\log(\hat{p}) & \text{if } y = 1 \\ -\log(1-\hat{p}) & \text{if } y = 0 \end{cases}$$

$$J(\boldsymbol{\theta}) = -\frac{1}{m}\sum_{i=1}^{m}\left[y^{(i)}log\left(\hat{p}^{(i)}\right) + \left(1-y^{(i)}\right)log\left(1-\hat{p}^{(i)}\right)\right]$$

$$\frac{\partial}{\partial\theta_j}\mathrm{J}(\boldsymbol{\theta}) = \frac{1}{m}\sum_{i=1}^{m}\left(\sigma\left(\boldsymbol{\theta}^T\mathbf{x}^{(i)}\right) - y^{(i)}\right)x_j^{(i)}$$

Where:

- $J(\theta) = NLL$ and is the function that we want to maximize for optimal result.
- **The optimization algorithm can be, again, the gradient descent**

Linear Models
Logistic Regression

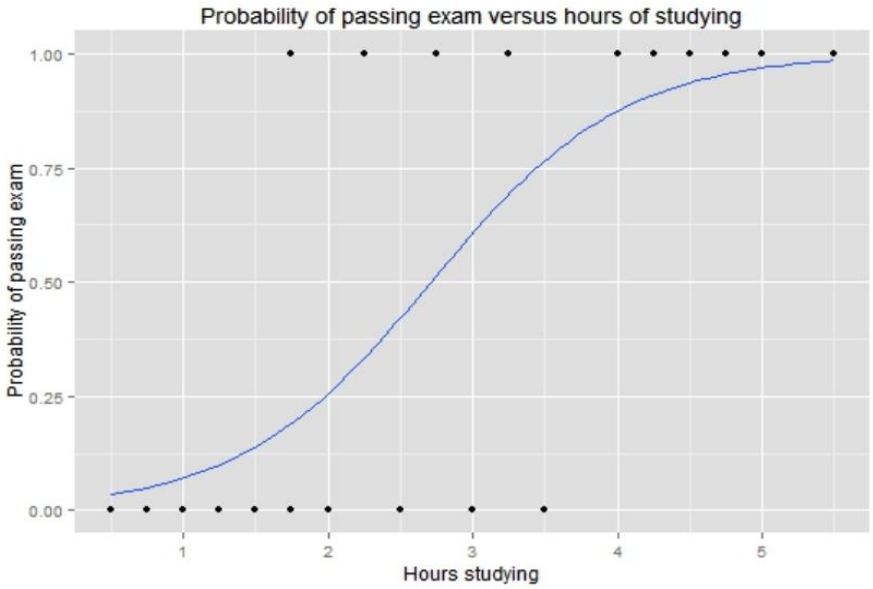
Example: Predicting if a Student Passes an Exame

Hours Studied	Passed
1	0
2	0
3	0
4	1
5	1
6	1

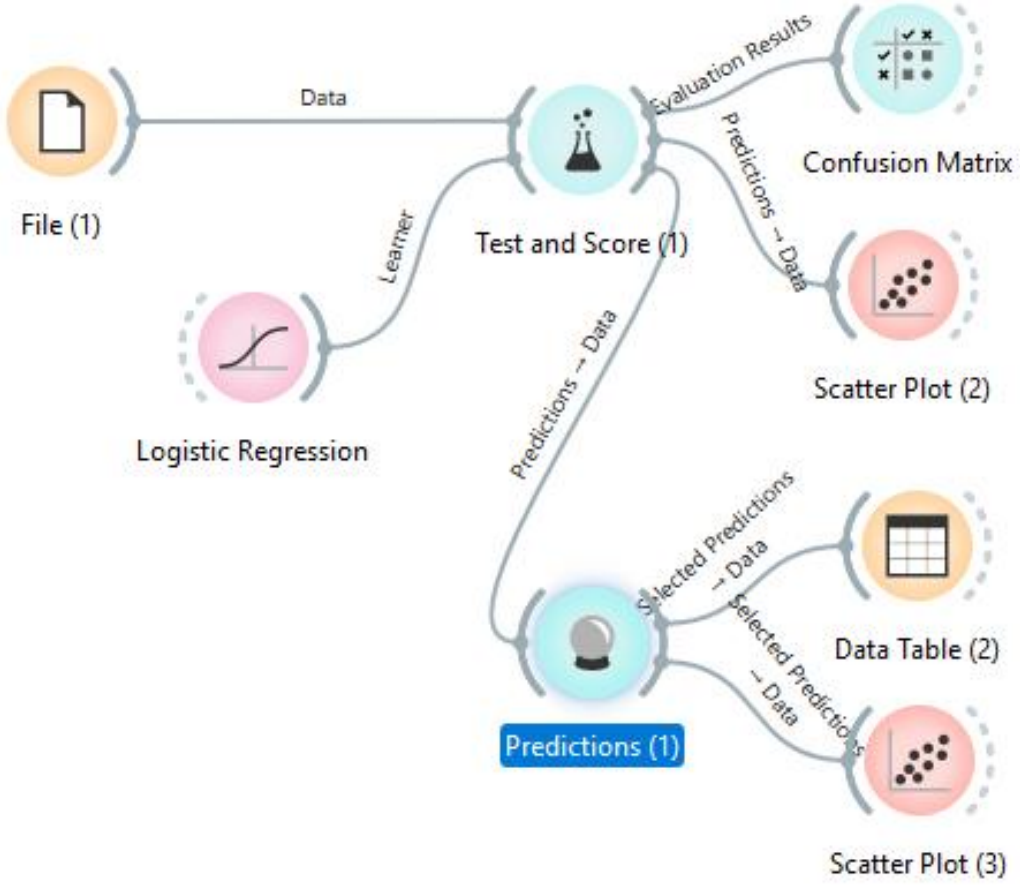
Intuition says, more hours you study more likely to pass. **But will not give a probability**

Fitting a sigmoid curve to the data, you will have a way to get a probability:

- 1 hour -> 5% chance
- 3 hour -> 25% chance
- 4 hour -> 60% chance



Linear Models
Logistic Regression

Orange Example:

- Two classes output: titanic.tab; heart_disease.tab
- Multiple class output: zoo.tab